

Ottimizzazione Combinatoria

Teoria dei Grafi: Problemi di Flusso su Rete

Daniele Vigo

(basate su slide di Marco A. Boschetti e Ugo Dal Lago)

Anno Accademico 2025/2026, versione 4.0 - Maggio 2026



Alma Mater Studiorum Università di Bologna
Dipartimento di Ingegneria dell'Energia Elettrica
e dell'Informazione "Guglielmo Marconi"
daniele.vigo@unibo.it

Outline

- ① Problemi di Flusso su Rete
 - Generalità
 - Definizione e Modelli Matematici
- ② Il Problema del Flusso Massimo: Algoritmi
 - Algoritmo di Ford-Fulkerson
 - Algoritmo di Edmonds-Karp
- ③ Il Problema del Flusso di Costo Minimo: Algoritmi
 - Cammini Minimi Successivi
 - Cancellazione di Cicli
- ④ Problemi di Accoppiamento

Problemi di Flusso su Rete

Grafi e Reti

- Molti problemi pratici sono modellati naturalmente da grafi:
 - Reti stradali: Nodi = incroci, punti notevoli; Archi: tratti stradali
 - Reti idrauliche: Nodi = bacini, giunzioni; Archi: condotte
 - Reti elettriche: Nodi = componenti, giunzioni; Archi: collegamenti
 - Reti di telecomunicazioni: Nodi = utenti, switch; Archi: collegamenti
 - Reti sociali: Nodi = individui, account; Archi: collegamenti, relazioni
- I grafi sono anche utilizzati in generale per modellare problemi in cui ci siano:
 - Elementi (Nodi) che hanno tra loro relazioni (Archi)
 - gli archi possono essere orientati o non orientati ed avere ulteriori caratteristiche (costo, capacità, ...)

Grafi e Reti (2)

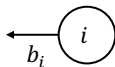
- Nel seguito introdurremo alcuni problemi di teoria dei grafi che si aggiungono ad altri già esaminati in corsi precedenti (cammini minimi, alberi di costo minimo, ...)
- In particolare esamineremo alcuni problemi di **Flusso su Rete**
- Sono in generale problemi “facili” per i quali illustreremo alcuni algoritmi polinomiali

Reti

- Una **Rete** è rappresentata da un grafo, generalmente **orientato (diretto)** $G = (N, A)$, in cui:
 - i vertici (nodi) $i \in N$, con $|N| = n$ sono punti di **ingresso**, **uscita** e **trasferimento**
 - gli archi $(i, j) \in A$, con $|A| = m$ sono i **collegamenti** tra i nodi
 - i collegamenti sono *orientati* dal vertice i (*coda* o *v. iniziale*) al vertice j (*testa* o *v. finale*)
- una **soluzione** è rappresentata da un **flusso** di oggetti tra i nodi attraverso gli archi della rete e può essere
 - **discreto**: ad esempio camion o pacchetti
 - **continuo**: ad esempio una quantità di fluido o di corrente

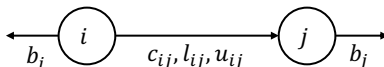
Nodi

- Ad ogni nodo $i \in N$ è associata un valore reale b_i , detto **sbilanciamento**, che può essere:
 - *Positivo*: ($D \subset N = \{i \in N : b_i > 0\}$)
 - Il nodo i è un nodo di uscita dalla rete e viene detto **destinazione**, **pozzo**, **nodo di output**
 - b_i è detto **domanda** di i .
 - *Negativo*: ($O \subset N = \{i \in N : b_i < 0\}$)
 - Il nodo i è un nodo di ingresso dalla rete e viene detto **origine**, **sorgente**, **nodo di input**
 - b_i è detto **offerta** di i .
 - *Nulla*: ($T \subset N = \{i \in N : b_i = 0\}$)
 - Il nodo i è detto **di trasferimento**
- $N = O \cup D \cup T$



Archi

- Ad ogni arco $(i,j) \in A$ sono associati:
 - Un **vertice iniziale** i ed in **vertice finale** j
 - Un **costo** c_{ij} , che indica quanto costa, per un'unità di flusso, utilizzare o attraversare il collegamento.
 - Una **capacità inferiore** l_{ij} , che indica un limite inferiore alla quantità di flusso che può fluire attraverso il collegamento.
 - Una **capacità superiore** u_{ij} , che indica un limite superiore alla quantità di flusso che può fluire attraverso il collegamento



Flusso sulla Rete

- Nei problemi di flusso, una **soluzione** non è altro che un flusso, ossia un assegnamento di valori reali agli archi di una rete $G = (N, A)$
- Ciò viene rappresentato da un insieme di *variabili di flusso* (o *flussi*) x_{ij} , ciascuna corrispondente ad un arco $(i, j) \in A$
- Il costo di un flusso non è nient'altro che il costo complessivo di tutti i flussi presenti nella rete:

$$\sum_{(i,j) \in A} c_{ij} x_{ij}$$

Vincoli

- **Domanda ed offerta globale si equivalgono:**

$$\sum_{i \in D} b_i = - \sum_{i \in O} b_i \iff \sum_{i \in N} b_i = 0$$

dove $D = \{i \in N : b_i > 0\}$ e $O = \{i \in N : b_i < 0\}$

- si può sempre costruire un rete equivalente che lo rispetti
 - se $\sum_{i \in D} b_i + \sum_{i \in O} b_i = \delta > 0$ basta creare una sorgente fittizia con domanda $-\delta$ collegata a costo 0 a tutti i nodi
 - se $\sum_{i \in D} b_i + \sum_{i \in O} b_i = \delta < 0$ basta creare un pozzo fittizio con domanda $-\delta$ collegato a costo 0 a tutti i nodi
- **il flusso deve essere ammissibile:**

$$l_{ij} \leq x_{ij} \leq u_{ij} \quad \forall (i,j) \in A$$

Vincoli (2)

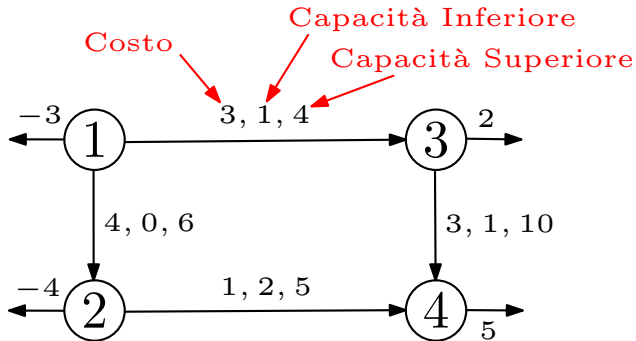
- Il flusso si conserva:

$$\sum_{(j,i) \in BS(i)} x_{ji} - \sum_{(i,j) \in FS(i)} x_{ij} = b_i \quad \forall i \in N$$

dove

- $BS(i) = \{(k,i) | (k,i) \in A\}$: detto **backward star** di i
- $FS(i) = \{(i,k) | (i,k) \in A\}$: detto **forward star** di i

Esempio di Rete



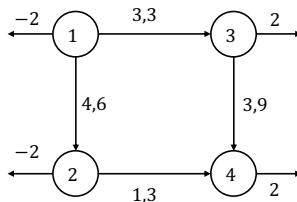
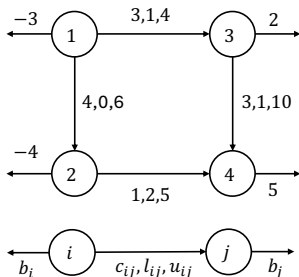
Ipotesi semplificative: capacità inferiore nulla

- Per semplificare problema e notazione supponiamo che
$$l_{ij} = 0 \quad \forall (i,j) \in A$$
- è sempre possibile farlo:
 - Data una rete G , si può sempre costruire una rete H equivalente a G e che abbia capacità inferiori nulle. Per ogni arco $(i,j) \in A$,
 - si sottrae la quantità l_{ij} a b_j e a u_{ij}
 - si aggiunge la quantità l_{ij} a b_i
 - alla funzione obiettivo bisognerà aggiungere

$$\sum_{(i,j) \in A} c_{ij} l_{ij}$$

- In altre parole, ad un flusso x_{ij} in H corrisponderà un flusso $x_{ij} + l_{ij}$ in G

Esempio di Rete senza l_{ij}

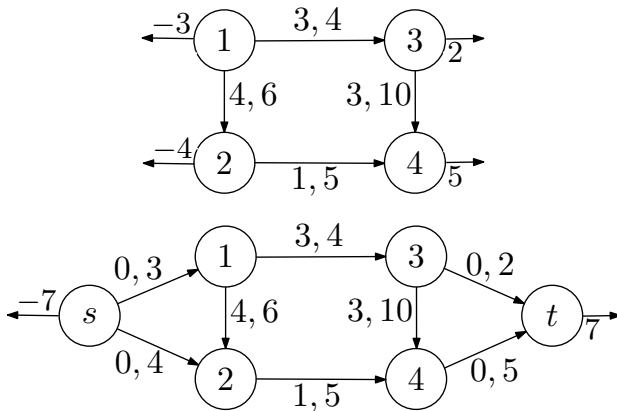


$$\begin{aligned}
 (1,2) \quad l_{12} &= 0: u_{12} = 6 - 0 = 6, b_1 = -3 + 0 = -3, b_2 = -4 - 0 = -4 \\
 (1,3) \quad l_{13} &= 1: u_{13} = 4 - 1 = 3, b_1 = -3 + 1 = -2, b_3 = 2 - 1 = 1 \\
 (2,4) \quad l_{24} &= 2: u_{24} = 5 - 2 = 3, b_2 = -4 + 2 = -2, b_4 = 5 - 2 = 3 \\
 (3,4) \quad l_{34} &= 1: u_{34} = 10 - 1 = 9, b_3 = 1 + 1 = 2, b_4 = 3 - 1 = 2
 \end{aligned}$$

Ipotesi semplificativa: una sola sorgente e pozzo

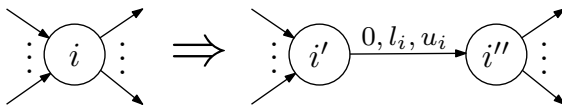
- Spesso, nel progetto di algoritmi per problemi di flusso, conviene assumere che vi sia una sola sorgente e un solo pozzo.
- Si può costruire una rete equivalente:
 - Aggiungendo due nodi fittizi, uno corrispondente all'unica sorgente, e l'altro all'unico pozzo.
 - Aggiungendo archi fittizi dalla sorgente a ciascuna delle sorgenti della rete di partenza. Tali archi avranno costo nullo, e capacità superiore pari all'inverso dello sbilanciamento della sorgente.
 - Aggiungendo archi fittizi da ciascuno dei pozzi della rete di partenza al pozzo. Tali archi avranno anch'essi costo nullo, ma capacità superiore pari allo sbilanciamento del pozzo.
 - Lo sbilanciamento dell'unica sorgente sarà uguale alla somma degli sbilanciamenti delle sorgenti. Similmente per il pozzo.

Ipotesi semplificativa: una sola sorgente e pozzo (2)



Capacità sui nodi

- Alcune volte è utile imporre che anche i nodi (e non solo gli archi) abbiano delle capacità, ossia che solo una quantità di flusso compresa nell'intervallo chiuso $[l_i, u_i]$ possa passare per il nodo $i \in N$
- è possibile costruire una rete equivalente con capacità solo sugli archi:
 - si sdoppia ogni nodo i con capacità in due nodi i', i'' , in modo che:
 - tutti gli archi entranti in i entrino in i'
 - tutti gli archi uscenti da i partano da i''
 - Si aggiunge un arco fittizio (i', i'') con costo nullo, capacità inferiore l_i e capacità superiore u_i



Il Problema del Flusso di Costo Minimo (MCF)

- Nel problema del *flusso di costo minimo* (o *minimum cost flow*, MCF):
 - Il costo del flusso è la funzione obiettivo, ovviamente da **minimizzare**.
 - Le capacità inferiori sono nulle.
- Il problema è formalizzabile facilmente in programmazione lineare:

$$\min cx$$

$$0 \leq x \leq u \quad Ex = b$$

dove

- $c \in \mathbb{R}^{|A|}$ è il **vettore dei costi**;
- $u \in \mathbb{R}^{|A|}$ è il **vettore delle capacità (superiori)**;
- $E \in \mathbb{R}^{|N| \times |A|}$ è una matrice di incidenza tra nodi e archi;
- $b \in \mathbb{R}^{|N|}$ è il **vettore degli sbilanciamenti**.

Definire il Problema — I

- Il **problema di flusso massimo** (o *maximum flow*, MF) è un problema di ottimizzazione su reti che può essere visto come una restrizione di MCF.
- Ciò che cambia è la funzione obiettivo: non vogliamo *minimizzare* i costi, ma *massimizzare* i flussi.
- i costi diventano irrilevanti (assumano $c_{ij} = 0 \forall (i,j) \in A$)
- Formalmente, data una rete $G = (N, A)$:
 - Fissiamo due nodi s (detto **sorgente**) e t (detto **destinazione**);
 - Vogliamo massimizzare il flusso da s a t , ossia trovare il **massimo** valore v tale che se $b_s = -v$, $b_t = v$ e $b_i = 0$ in tutti gli altri casi, allora esiste un flusso ammissibile (di qualsiasi costo).
- Un valore v *ammissibile* per il problema di cui sopra si dice **valore** del flusso x .
- Il problema è formalizzabile direttamente in PL.

Definire il Problema — II

$\max v$

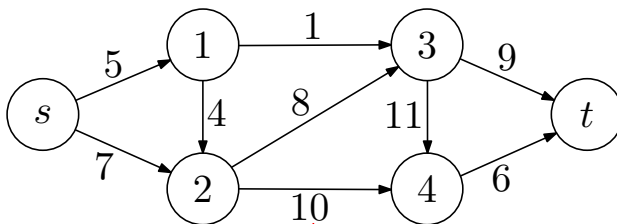
$$\sum_{(j,s) \in BS(s)} x_{js} + v = \sum_{(s,j) \in FS(s)} x_{sj};$$

$$\sum_{(j,i) \in BS(i)} x_{ji} - \sum_{(i,j) \in FS(i)} x_{ij} = 0, \quad i \in N - \{s, t\};$$

$$\sum_{(j,t) \in BS(t)} x_{jt} = \sum_{(t,j) \in FS(t)} x_{tj} + v;$$

$$0 \leq x_{ij} \leq u_{ij}, \quad (i,j) \in A.$$

Esempio

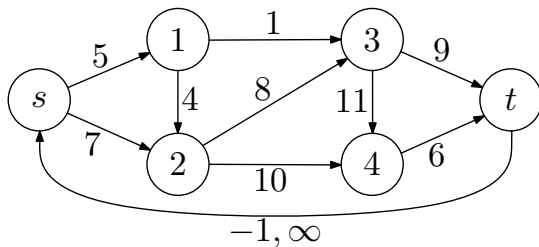
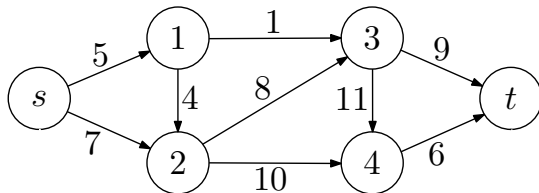


Capacità Superiore

Massimo Flusso e MCF

- Il problema MF può essere visto come un caso particolare di MCF:
 - I costi sono nulli;
 - Gli sbilanciamenti sono nulli;
 - Si aggiunge però un arco fittizio da t a s con costo -1 e capacità infinita.
- Graficamente:

Massimo Flusso e MCF (2)



Algoritmi Primali e Duali

- Algoritmo PRIMALE:

- Un algoritmo **primale** parte da una soluzione **ammissibile** per il primale ma non ottima (quindi **inammissibile** per il duale)
- ad ogni iterazione costruisce una nuova soluzione **ammissibile** e **migliorante**
- termina quando la soluzione non è ulteriormente migliorabile (è quindi **ammissibile** per il duale)

- Algoritmo DUALE:

- Un algoritmo **duale** parte da una soluzione **non ammissibile** per il primale ma più che ottima (quindi **ammissibile** per il duale)
- ad ogni iterazione costruisce una nuova soluzione **meno inammissibile** e **peggiore**
- termina quando la soluzione è ammissibile per il primale

Il Problema del Flusso Massimo: Algoritmi

Algoritmi per Flusso Massimo

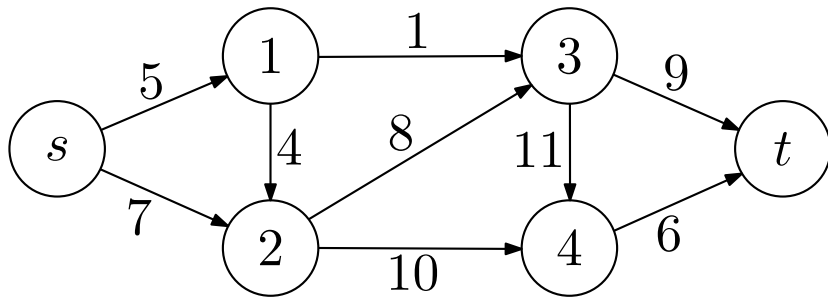
- Esistono numerosi algoritmi per determinare il flusso massimo di una rete
- Esamineremo uno dei primi e più semplici di questi, proposto da L.R. Ford e D.R. Fulkerson nel 1956.
- E' un ulteriore esempio di algoritmo PRIMALE
- Supponiamo che la rete abbia un solo nodo sorgente ed un solo nodo pozzo come discusso precedentemente

Concetti preliminari: Tagli

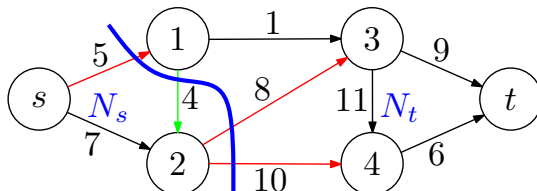
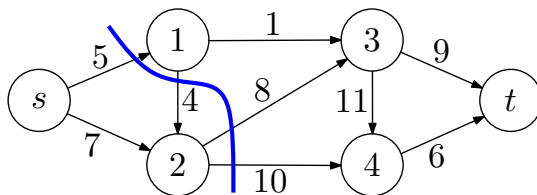
- Un **taglio** in una rete $G = (N, A)$ è dato da una coppia (N', N'') di sottoinsiemi di N tali che $N' \cap N'' = \emptyset$ e $N' \cup N'' = N$.
- Un (s, t) -**taglio** in una rete G è un taglio (N_s, N_t) dove $s \in N_s$ e $t \in N_t$.
- Dato un (s, t) -taglio in $G = (N, A)$, indichiamo con $A^+(N_s, N_t)$ e $A^-(N_s, N_t)$ i seguenti sottoinsiemi di A :

$$\begin{aligned} \text{archi avanti : } A^+(N_s, N_t) &= \{(i, j) \in A \mid i \in N_s \wedge j \in N_t\}; \\ \text{archi indietro : } A^-(N_s, N_t) &= \{(i, j) \in A \mid i \in N_t \wedge j \in N_s\}. \end{aligned}$$

Tagli — Esempio



Tagli — Esempio (2)



Tagli — Proprietà

Lemma

Per ogni (s, t) -taglio (N_s, N_t) e ogni flusso ammissibile x con valore v :

1. $v = \sum_{(i,j) \in A^+(N_s, N_t)} x_{ij} - \sum_{(i,j) \in A^-(N_s, N_t)} x_{ij}$;
2. $v \leq \sum_{(i,j) \in A^+(N_s, N_t)} u_{ij}$.

Tagli — Proprietà (2)

Dimostrazione.

1. Osserviamo che:

$$\begin{aligned}
 v &= \sum_{(s,j) \in A} x_{sj} - \sum_{(i,s) \in A} x_{is} = \sum_{k \in N_s} \left(\sum_{(k,j) \in A} x_{kj} - \sum_{(i,k) \in A} x_{ik} \right) \\
 &= \sum_{(i,j) \in A^+(N_s, N_t)} x_{ij} - \sum_{(i,j) \in A^-(N_s, N_t)} x_{ij}
 \end{aligned}$$

2. E' una semplice conseguenza del punto 1.



Tagli — Proprietà

- Diamo un nome alle due quantità che abbiamo studiato nel Lemma precedente:
 - La quantità $\sum_{(i,j) \in A^+(N_s, N_t)} x_{ij} - \sum_{(i,j) \in A^-(N_s, N_t)} x_{ij}$ è detta **flusso del taglio** (N_s, N_t) , ed è indicata con $x(N_s, N_t)$.
 - La quantità $\sum_{(i,j) \in A^+(N_s, N_t)} u_{ij}$ è detta **capacità del taglio** (N_s, N_t) , ed è indicata con $u(N_s, N_t)$.
- Quello che ci dice il Lemma precedente è che

$$v = x(N_s, N_t) \leq u(N_s, N_t).$$

- In altre parole: *il valore di un flusso ammissibile è sempre minore o uguale della capacità di qualunque taglio.*
- Ma esiste un taglio con capacità **identica** al valore di un flusso ammissibile (che quindi sarà massimo)?

Grafo Residuo

- Dati una rete $G = (N_G, A_G)$ e un flusso ammissibile x , il **grafo residuo** G_x è il *multigrafo* (N_{G_x}, A_{G_x}) tale che:
 - $N_{G_x} = N_G$;
 - Gli archi in A_{G_x} sono di due tipi:
 - Per ogni arco $(i, j) \in A_G$ tale che $x_{ij} < u_{ij}$ esiste un arco **da i a j** in G_x (detto **arco concorde** o **arco avanti**);
 - Per ogni arco $(i, j) \in A_G$ tale che $x_{ij} > 0$ esiste un arco **da j a i** in G_x (detto **arco discorde** o **arco indietro**).
 - Osserviamo come in N_{G_x} ci possano essere *due* archi da uno stesso nodo i ad uno stesso nodo j .

Cammini Aumentanti

- Un **cammino aumentante** P in una rete G rispetto a x non è nient'altro che un cammino semplice e orientato da s a t in G_x .
 - Sia P^+ l'insieme degli archi *concordi* in P , e P^- l'insieme dei suoi archi *discordi*.
- Dato un cammino aumentante P rispetto a x , definiamo la **capacità** di P rispetto a x come

$$\theta(P, x) = \min\{ \min\{u_{ij} - x_{ij} \mid (i, j) \in P^+\}, \\ \min\{x_{ij} \mid (j, i) \in P^-\} \}.$$

Cammini Aumentanti (2)

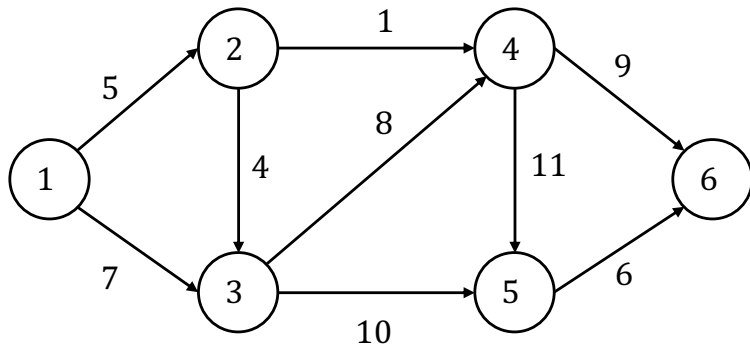
- Dato un flusso x , un cammino P in G_x e un reale θ , definiamo $x(P, \theta)$ il flusso definito come segue:

$$(x(P, \theta))_{ij} = \begin{cases} x_{ij} + \theta & \text{se } (i, j) \in P^+; \\ x_{ij} - \theta & \text{se } (j, i) \in P^-; \\ x_{ij} & \text{altrimenti.} \end{cases}$$

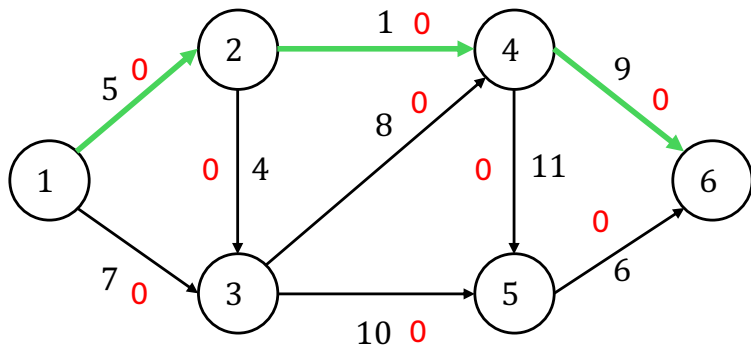
Algoritmo di Ford-Fulkerson

1. $x \leftarrow 0$;
2. Costruisci G_x e determina se G_x ha un cammino aumentante P . In caso P non esista, termina e restituisci x ;
3. $x \leftarrow x(P, \theta(P, x))$
4. Ritorna al punto 2.

Algoritmo di Ford-Fulkerson: Esempio

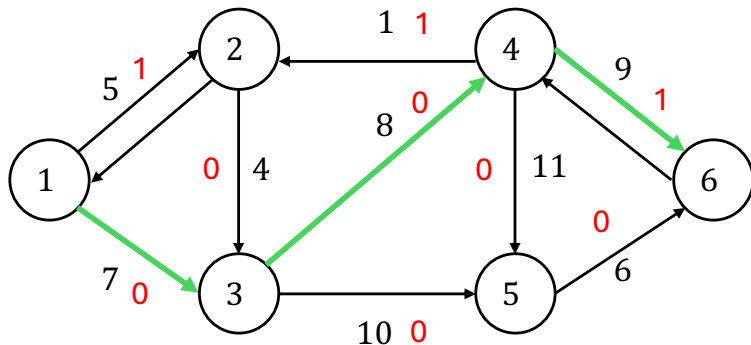


Algoritmo di Ford-Fulkerson: Esempio (2)



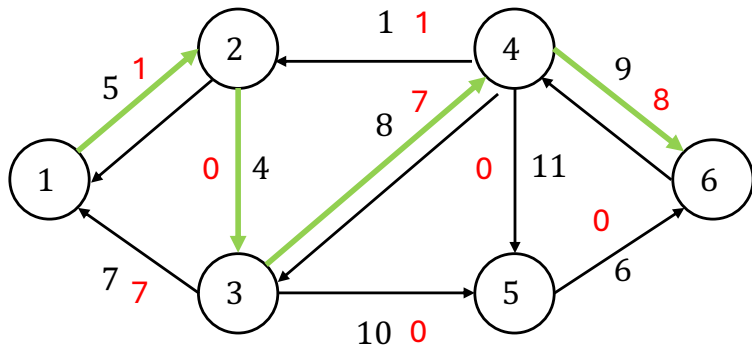
$$G_x \equiv G, P = \{1, 2, 4, 6\}, \theta(P, x) = 1$$

Algoritmo di Ford-Fulkerson: Esempio (3)



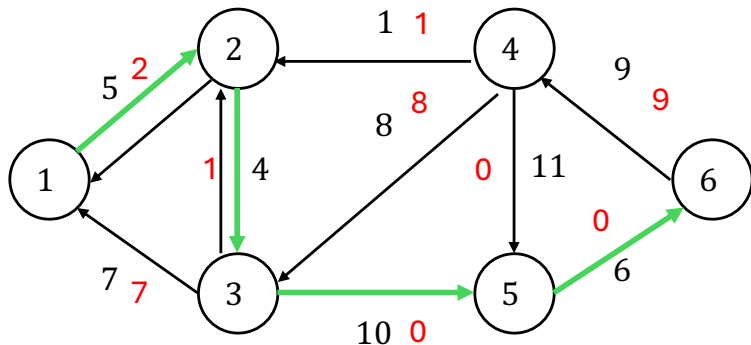
$$G_x, P = \{1, 3, 4, 6\}, \theta(P, x) = 7$$

Algoritmo di Ford-Fulkerson: Esempio (4)



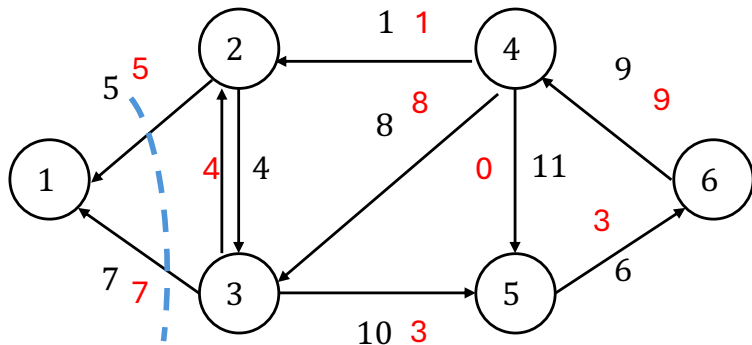
$$G_x, P = \{1, 2, 3, 4, 6\}, \theta(P, x) = 1$$

Algoritmo di Ford-Fulkerson: Esempio (5)



$$G_x, P = \{1, 2, 3, 5, 6\}, \theta(P, x) = 3$$

Algoritmo di Ford-Fulkerson: Esempio (6)



Cammino Aumentante in G_x

- Nella ricerca del cammino aumentante non è necessario costruire esplicitamente G_x
- è sufficiente associare ad ogni vertice $i \in N$ due etichette:
 - π_i : predecessore di i nel cammino da s ,
 - θ_i : massimo flusso aggiuntivo inviabile fino a i
 - se $\pi_i > 0$, usato l'arco avanti (π_i, i) , $\theta_i = \min\{\theta_{\pi_i}, u_{\pi_i, i} - x_{\pi_i, i}\}$
 - se $\pi_i < 0$, usato l'arco indietro $(i, |\pi_i|)$, $\theta_i = \min\{\theta_{|\pi_i|}, x_{i, |\pi_i|}\}$
- ovviamente gli etichettamenti in avanti o indietro devono essere compatibili con l'esistenza degli archi corrispondenti in G_x , ossia dato i
 - si può usare l'arco avanti (i, j) se j non ancora raggiunto e $u_{ij} - x_{ij} > 0$
 - si può usare l'arco indietro (j, i) se j non ancora raggiunto e $x_{ji} > 0$
- E' infine necessario mantenere gli indici dei vertici da cui espandere in un insieme Q

Cammino Aumentante in G_x (2)

CAMMINO_AUMENTANTE($G, x, found, \pi, \theta$)

1. **for each** $i \in N$ **do** $\pi_i = nil, \theta_i = 0$
2. $Q = s, \pi_s = s, \theta_s = +\infty, found = false$
3. **While** $Q \neq \emptyset$ **do**
4. Estrai i da Q
5. **for each** $j \in FS(i)$ **do**
6. **if** $\pi_j = nil \wedge (u_{ij} - x_{ij}) > 0$ **then**
7. $\pi_j = i, \theta_j = \min\{\theta_i, u_{ij} - x_{ij}\}, Q = Q \cup \{j\}$
8. **for each** $j \in BS(i)$ **do**
9. **if** $\pi_j = nil \wedge x_{ji} > 0$ **then**
10. $\pi_j = -i, \theta_j = \min\{\theta_i, x_{ji}\}, Q = Q \cup \{j\}$
11. **if** $\pi_t \neq nil$ **then**
12. $found = true, \text{break}$
13. **endWhile**

Algoritmo di Ford-Fulkerson (v.2)

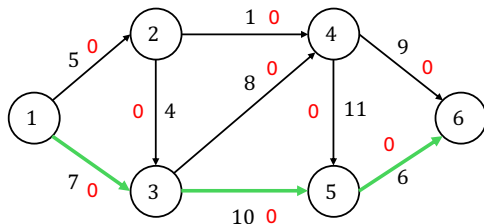
1. $x \leftarrow 0, v = 0;$
2. **repeat**
3. CAMMINO_AUMENTANTE($G, x, found, \pi, \theta$)
4. **if** $found = true$ **then**
5. modifica x di θ_t lungo il cammino definito da $\pi, v = v + \theta_t$
6. **until** $found = false$

Algoritmo di Ford-Fulkerson : implementazione

- Ford-Fulkerson è in realtà un metodo...
- possono essere ottenute implementazioni diverse a seconda di come viene gestita la visita di G_x per trovare il cammino aumentante
- Diverse modalità di visita possono essere ottenute definendo in modo opportuno l'insieme Q dei nodi da espandere
 - Se Q è una **coda** (FIFO) la visita avviene *per ampiezza* (BFS) , la ricerca del cammino aumentante è più lenta e restituisce un cammino di lunghezza minima (minimo n. di archi)
 - Se Q è una **pila** (LIFO) la visita avviene *per profondità* (DFS) , la ricerca del cammino aumentante è più veloce e restituisce un cammino di lunghezza qualunque

Algoritmo di Ford-Fulkerson v.2: Esempio

- supponiamo dapprima Q sia una pila (stack) $\Rightarrow$ DFS

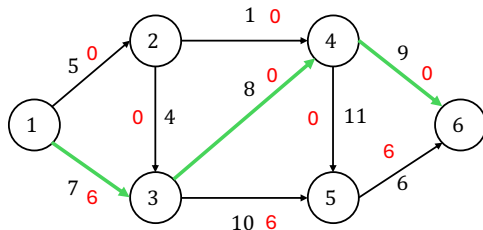


```

Iterazione : 1
espando da vertice 1
raggiungo avanti : 2 theta 5
raggiungo avanti : 3 theta 7
espando da vertice 3
raggiungo avanti : 4 theta 7
raggiungo avanti : 5 theta 7
espando da vertice 5
raggiungo avanti : 6 theta 6

Pred : 1 1 1 3 3 5
Theta : Inf 5 7 7 7 6
trovato cammino aumentante : 6, 5, 3, 1,
valore 6,
nuovo flusso 6
  
```

Algoritmo di Ford-Fulkerson v.2: Esempio (2)



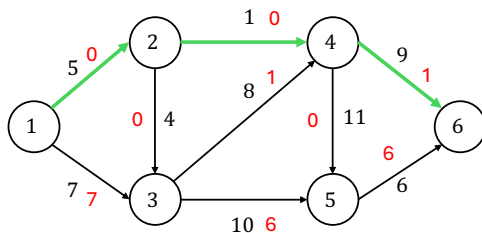
```

Iterazione : 2
espando da vertice 1
raggiungo avanti : 2 theta 5
raggiungo avanti : 3 theta 1
espando da vertice 3
raggiungo avanti : 4 theta 1
raggiungo avanti : 5 theta 1
espando da vertice 5
espando da vertice 4
raggiungo avanti : 6 theta 1
  
```

```

Pred : 1 1 1 3 3 4
Theta : Inf 5 1 1 1 1
trovato cammino aumentante : 6, 4, 3, 1,
valore 1, nuovo flusso 7
  
```


Algoritmo di Ford-Fulkerson v.2: Esempio (3)



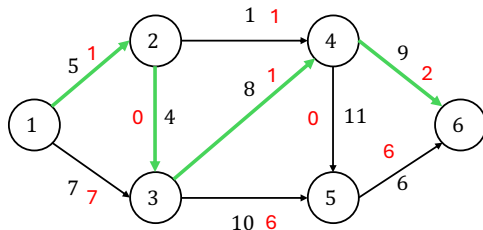
```

Iterazione : 3
espando da vertice 1
raggiungo avanti : 2 theta 5
espando da vertice 2
raggiungo avanti : 3 theta 4
raggiungo avanti : 4 theta 1
espando da vertice 4
raggiungo avanti : 5 theta 1
raggiungo avanti : 6 theta 1
  
```

```

Pred : 1 1 2 2 4 4
Theta : Inf 5 4 1 1 1
trovato cammino aumentante : 6, 4, 2, 1,
valore 1, nuovo flusso 8
  
```

Algoritmo di Ford-Fulkerson v.2: Esempio (4)



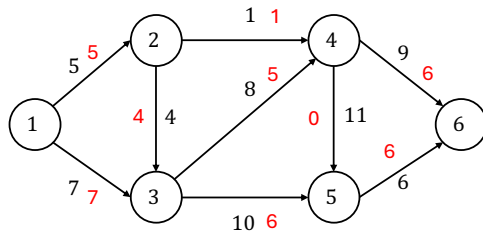
```

Iterazione : 4
espando da vertice 1
raggiungo avanti : 2 theta 4
espando da vertice 2
raggiungo avanti : 3 theta 4
espando da vertice 3
raggiungo avanti : 4 theta 4
raggiungo avanti : 5 theta 4
espando da vertice 5
espando da vertice 4
raggiungo avanti : 6 theta 4
  
```

```

Pred : 1 1 2 3 3 4
Theta : Inf 4 4 4 4 4
trovato cammino aumentante : 6, 4, 3, 2, 1,
valore 4, nuovo flusso 12
  
```

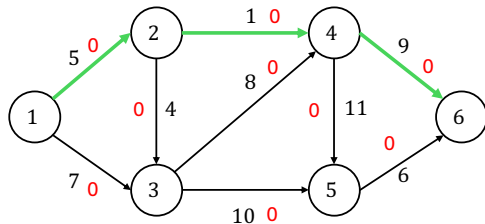
Algoritmo di Ford-Fulkerson v.2: Esempio (5)



Iterazione : 5
 espando da vertice 1
 non trovato cammino aumentante,
 vertici raggiunti: 1

Algoritmo di Ford-Fulkerson v.2: Esempio

- Supponiamo ora che Q sia una coda ordinata



```

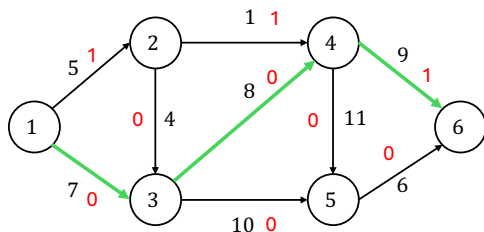
Iterazione : 1
espando da vertice 1
raggiungo avanti : 2 theta 5
raggiungo avanti : 3 theta 7
espando da vertice 2
raggiungo avanti : 4 theta 1
espando da vertice 3
raggiungo avanti : 5 theta 7
espando da vertice 4
raggiungo avanti : 6 theta 1
  
```

Pred : 1 1 1 2 3 4

Theta : Inf 5 7 1 7 1

trovato cammino aumentante : 6, 4, 2, 1, valore 1,
nuovo flusso 1

Algoritmo di Ford-Fulkerson v.2: Esempio (2)



Iterazione : 2

espando da vertice 1

raggiungo avanti : 2 theta 4

raggiungo avanti : 3 theta 7

espando da vertice 2

espando da vertice 3

raggiungo avanti : 4 theta 7

raggiungo avanti : 5 theta 7

espando da vertice 4

raggiungo avanti : 6 theta 7

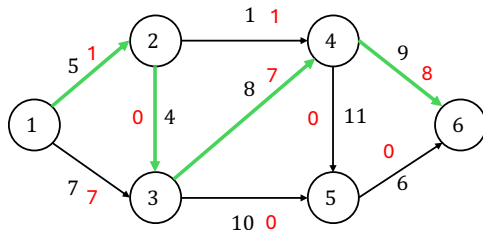
Pred : 1 1 1 3 3 4

Theta : Inf 4 7 7 7 7

trovato cammino aumentante : 6, 4, 3, 1, valore 7

nuovo flusso 8

Algoritmo di Ford-Fulkerson v.2: Esempio (3)



```

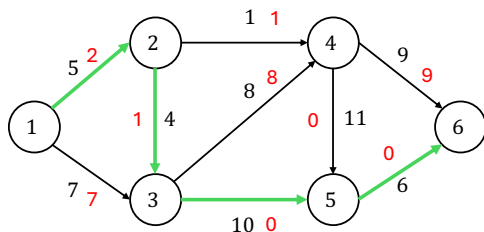
Iterazione : 3
espando da vertice 1
raggiungo avanti : 2 theta 4
espando da vertice 2
raggiungo avanti : 3 theta 4
espando da vertice 3
raggiungo avanti : 4 theta 1
raggiungo avanti : 5 theta 4
espando da vertice 4
raggiungo avanti : 6 theta 1
  
```

```
Pred : 1 1 2 3 3 4
```

```
Theta : Inf 4 4 1 4 1
```

```
trovato cammino aumentante : 6, 4, 3, 2, 1, valo
nuovo flusso 9
```

Algoritmo di Ford-Fulkerson v.2: Esempio (4)



Iterazione : 4

espando da vertice 1

raggiungo avanti : 2 theta 3

espando da vertice 2

raggiungo avanti : 3 theta 3

espando da vertice 3

raggiungo avanti : 5 theta 3

espando da vertice 5

raggiungo avanti : 6 theta 3

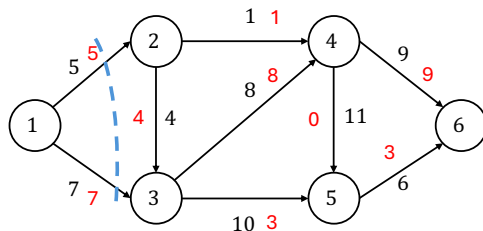
Pred : 1 1 2 -1000 3 5

Theta : Inf 3 3 0 3 3

trovato cammino aumentante : 6, 5, 3, 2, 1, valore

nuovo flusso 12

Algoritmo di Ford-Fulkerson v.2: Esempio (5)



Iterazione : 5
 espando da vertice 1
 non trovato cammino aumentante,
 vertici raggiunti: 1

Ford-Fulkerson — Correttezza

Lemma

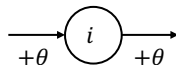
Se x è ammissibile, allora anche $x(P, \theta(P, x))$ è ammissibile.

Ford-Fulkerson — Correttezza (2)

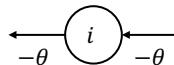
Dimostrazione.

- su ogni arco avanti di P , x aumenta di θ , su ogni arco indietro diminuisce di θ
- ⇒ per come è definito θ le capacità sono tutte rispettate
- Se x è ammissibile rispetta la conservazione di flusso ad ogni nodo
- per ogni nodo non appartenente a P i flussi non cambiano: conservazione OK!
- per ogni nodo appartenente a P i flussi cambiano di:

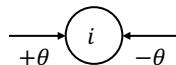
due archi "avanti"



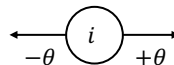
due archi "indietro"



Discordi 1



Discordi 2



Ford-Fulkerson — Correttezza

Lemma

Se G_x non ha cammini aumentanti, allora esiste un taglio di capacità pari a v .

Ford-Fulkerson — Correttezza (2)

Dimostrazione.

Basta considerare Il taglio (N_s, N_t) , dove N_s contiene tutti e soli i nodi raggiungibili da s in G_x (e $N_t = N - N_s$). Infatti

$$\begin{aligned}
 v = x(N_s, N_t) &= \sum_{(i,j) \in A^+(N_s, N_t)} x_{ij} - \sum_{(i,j) \in A^-(N_s, N_t)} x_{ij} \\
 &= \sum_{(i,j) \in A^+(N_s, N_t)} u_{ij} - \sum_{(i,j) \in A^-(N_s, N_t)} 0 \\
 &= u(N_s, N_t)
 \end{aligned}$$



Ford-Fulkerson — Correttezza

Teorema (Correttezza)

Se l'algoritmo di Ford-Fulkerson termina, allora il flusso x è un flusso massimo.

Dimostrazione.

- Se Ford-Fulkerson termina, allora G_x non ha cammini aumentanti.
- Ma quindi esiste un taglio di capacità pari a v .
- E a questo punto v non può essere che massimo, perché se non lo fosse avremmo un taglio di capacità inferiore al valore di un flusso ammissibile.



Max-Flow Min-Cut

Teorema

Il valore del massimo flusso è uguale alla minima capacità dei tagli.

Dimostrazione.

- Basta dimostrare che il valore del massimo flusso è maggiore o uguale alla capacità di *un* taglio.
- Ma se x è ammissibile e massimo, G_x non ha cammini aumentanti, e quindi esiste un taglio di capacità pari a v .



Ford-Fulkerson — Complessità

Teorema

Se le capacità di G sono numeri interi, allora esiste almeno un flusso intero massimo.

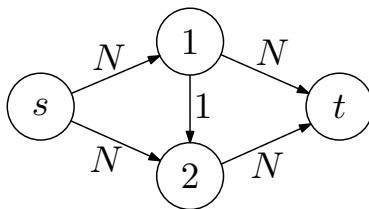
Dimostrazione.

- Se le capacità sono intere, allora il flusso massimo sarà al più nU dove $U = \max\{u_{ij} \mid (i,j) \in A\}$.
- Si parte da un flusso intero, e l'interezza è preservata, perché per ogni cammino aumentante P , $\theta(P, x)$ è un numero intero.
- Di conseguenza, il valore del flusso aumenta almeno di 1 ad ogni iterazione.
- L'algoritmo terminerà quindi dopo al più nU iterazioni.



Ford-Fulkerson — Complessità

- La dimostrazione del teorema precedente ci dice che se le capacità sono intere, allora la complessità è $O(mnU)$, ovvero solo **pseudopolinomiale** nella dimensione della rete
- Controesempio alla polinomialità:



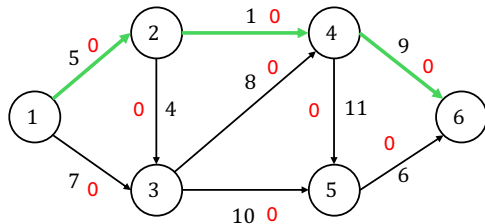
- Cammino 1: $s, 1, 2, t$, flusso = 1
 - Cammino 2: $s, 2, 1, t$, flusso = 1 ...
- In assenza del vincolo di interezza, non si può dire molto sulla complessità dell'algoritmo.

L'Algoritmo di Edmonds-Karp

- E' possibile rendere la complessità di Ford-Fulkerson **polinomiale** in tempo?
- Un modo consiste nel lasciare l'algoritmo così com'è, ma agire sul **modo** in cui i cammini aumentanti vengono scoperti, ossia sull'algoritmo usato per **visitare** il grafo residuo G_x .
- L'**Algoritmo di Edmonds-Karp** non è nient'altro che l'algoritmo di Ford-Fulkerson dove, però, la ricerca del cammino aumentante viene eseguita visitando **in ampiezza** (BFS) il grafo residuo G_x .
- Notiamo che in questo modo i cammini aumentanti saranno sempre cammini **di lunghezza minima**.

Algoritmo di Edmonds-Karp: Esempio

- Supponiamo che Q sia una coda FIFO



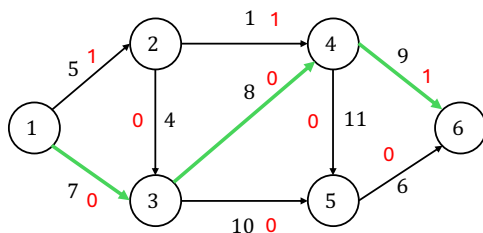
```

Iterazione : 1
espando da vertice 1
raggiungo avanti : 2 theta 5
raggiungo avanti : 3 theta 7
espando da vertice 2
raggiungo avanti : 4 theta 1
espando da vertice 3
raggiungo avanti : 5 theta 7
espando da vertice 4
raggiungo avanti : 6 theta 1
  
```

```

Pred : 1 1 1 2 3 4
Theta : Inf 5 7 1 7 1
trovato cammino aumentante : 6, 4, 2, 1,
valore 1, nuovo flusso 1
  
```

Algoritmo di Edmonds-Karp: Esempio (2)



Iterazione : 2

espando da vertice 1

raggiungo avanti : 2 theta 4

raggiungo avanti : 3 theta 7

espando da vertice 2

espando da vertice 3

raggiungo avanti : 4 theta 7

raggiungo avanti : 5 theta 7

espando da vertice 4

raggiungo avanti : 6 theta 7

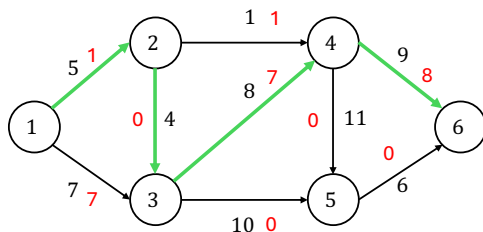
Pred : 1 1 1 3 3 4

Theta : Inf 4 7 7 7 7

trovato cammino aumentante : 6, 4, 3, 1,

valore 7, nuovo flusso 8

Algoritmo di Edmonds-Karp: Esempio (3)



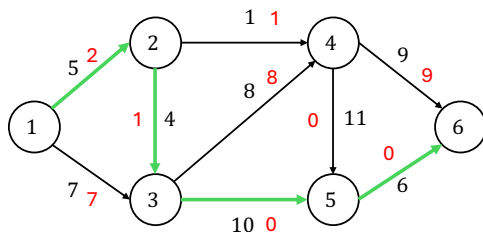
```

Iterazione : 3
espando da vertice 1
raggiungo avanti : 2 theta 4
espando da vertice 2
raggiungo avanti : 3 theta 4
espando da vertice 3
raggiungo avanti : 4 theta 1
raggiungo avanti : 5 theta 4
espando da vertice 4
raggiungo avanti : 6 theta 1
  
```

```

Pred : 1 1 2 3 3 4
Theta : Inf 4 4 1 4 1
trovato cammino aumentante : 6, 4, 3, 2, 1,
valore 1, nuovo flusso 9
  
```

Algoritmo di Edmonds-Karp: Esempio (4)



Iterazione : 4

espando da vertice 1

raggiungo avanti : 2 theta 3

espando da vertice 2

raggiungo avanti : 3 theta 3

espando da vertice 3

raggiungo avanti : 5 theta 3

espando da vertice 5

raggiungo avanti : 6 theta 3

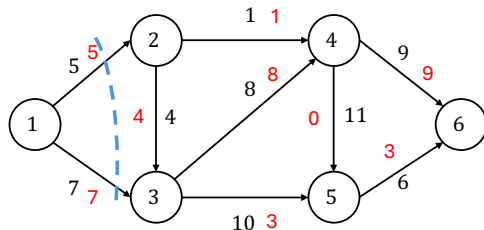
Pred : 1 1 2 -1000 3 5

Theta : Inf 3 3 0 3 3

trovato cammino aumentante : 6, 5, 3, 2, 1,

valore 3, nuovo flusso 12

Algoritmo di Edmonds-Karp: Esempio (5)



Iterazione : 5
 espando da vertice 1
 non trovato cammino aumentante,
 vertici raggiunti: 1

Edmonds-Karp — Proprietà

- EK è trivialmente **corretto**, essendo nient'altro che un caso particolare di FF.
- Studiarne la **complessità**, invece, risulta più difficile:
 - Si procede osservando che, *se in FF i cammini aumentanti sono di lunghezza minima*, allora la distanza di un generico nodo i dalla sorgente s in G_x **non può diminuire**.
 - Da ciò si deduce che il numero di iterazioni di EK non può, asintoticamente essere più grande di $n \cdot m$.

Edmonds-Karp — Proprietà

- Data una rete G , un flusso ammissibile x e due nodi i, j in G , indichiamo con $\delta_x(i, j)$ la distanza tra i e j nel grafo residuo G_x .

Lemma

Se, durante l'esecuzione di EK, il flusso y è ottenuto da x tramite un'operazione di aumento del flusso in un cammino aumentante, allora per ogni nodo $i \in N$, vale che $\delta_x(s, i) \leq \delta_y(s, i)$.

- La prova è molto interessante ed ingegnosa. Non abbiamo purtroppo tempo di vederla in dettaglio. Gli studenti interessati possono consultare le note.

Edmonds-Karp — Proprietà

Teorema

Il numero di iterazioni di EK è $O(nm)$, quindi la sua complessità è $O(nm^2)$.

Dimostrazione.

- Un arco (i,j) in G_x è detto *critico* per un cammino aumentante P se la sua capacità (ossia $u_{ij} - x_{ij}$ se è concorde o x_{ji} se discorde) è uguale a $\theta(P,x)$.
 - Dopo l'aumento del flusso lungo P , l'arco (i,j) sparisce dal grafo residuo.
 - In ogni cammino aumentante, esiste almeno un arco critico.
- Dati i,j connessi da un arco in A , quante volte è possibile che (i,j) sia arco critico? Si può dimostrare che tale numero è al più $O(n)$, e visto che di tali coppie ne esistono al più $O(m)$, in totale potremo avere al più $O(nm)$ iterazioni. Nel seguito dimostriamo proprio il limite $O(n)$ al numero di volte in cui (i,j) può diventare critico.

- Quando (i,j) diventa critico la prima volta, deve valere che

$$\delta_x(s,j) = \delta_x(s,i) + 1,$$

dove x è il flusso, e a quel punto sparisce dal grafo residuo.

- L'unico modo per ricomparirvi è fare in modo che il flusso (reale o virtuale) da i a j *diminuisca*, e questo vuol dire che

$$\delta_y(s,i) = \delta_y(s,j) + 1,$$

dove y è il flusso.

- Dunque:

$$\delta_y(s,i) = \delta_y(s,j) + 1 \geq \delta_x(s,j) + 1 = \delta_x(s,i) + 2.$$

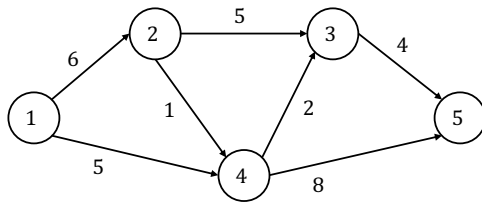
- Di conseguenza, da un momento in cui (i,j) diventa critico al successivo, la sua distanza da s aumenta di almeno 2. Siccome tale distanza non può essere superiore a n , il numero di volte in cui (i,j) può diventare critico è lineare in n .



Ci Sono Solo FF e EK?

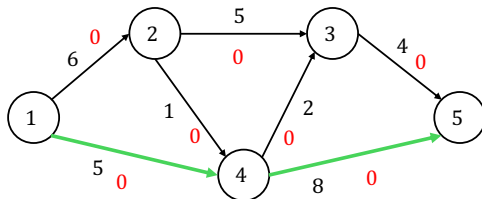
<i>Algoritmo</i>	<i>Complessità</i>
FF	$O(nmU)$
EK	$O(nm^2)$
Dinic	$O(n^2m)$
MKM	$O(n^3)$
Goldberg-Tarjan	$O(n^2m)$
KRT	$O(nm \log \frac{m}{n \log n})$
Orlin+KRT	$O(nm)$

Flusso Massimo: Esempio 2



Algoritmo di Ford Fulkerson: Esempio 2

- Supponiamo che Q sia una coda FIFO



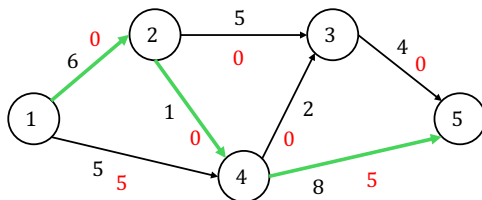
```

Iterazione : 1
espando da vertice 1
raggiungo avanti : 2 theta 6
raggiungo avanti : 4 theta 5
espando da vertice 4
raggiungo avanti : 3 theta 2
raggiungo avanti : 5 theta 5
  
```

```

Pred : 1 1 4 1 4
Theta : Inf 6 2 5 5
trovato cammino aumentante : 5, 4, 1,
valore 5, nuovo flusso 5
  
```

Algoritmo di Ford Fulkerson: Esempio 2 (2)



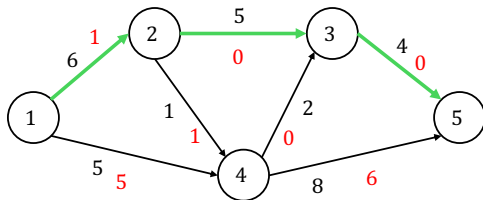
```

Iterazione : 2
espando da vertice 1
raggiungo avanti : 2 theta 6
espando da vertice 2
raggiungo avanti : 3 theta 5
raggiungo avanti : 4 theta 1
espando da vertice 4
raggiungo avanti : 5 theta 1
  
```

```

Pred : 1 1 2 2 4
Theta : Inf 6 5 1 1
trovato cammino aumentante : 5, 4, 2, 1,
valore 1, nuovo flusso 6
  
```

Algoritmo di Ford Fulkerson: Esempio 2 (3)

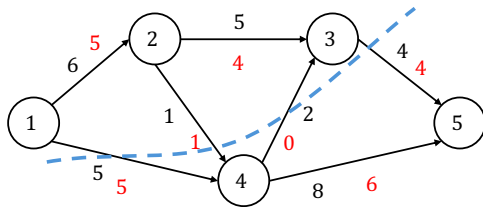


Iterazione : 3
 espando da vertice 1
 raggiungo avanti : 2 theta 5
 espando da vertice 2
 raggiungo avanti : 3 theta 5
 espando da vertice 3
 raggiungo avanti : 5 theta 4

Pred : 1 1 2 -1000 3

Theta : Inf 5 5 0 4

trovato cammino aumentante : 5, 3, 2, 1,
 valore 4, nuovo flusso 10



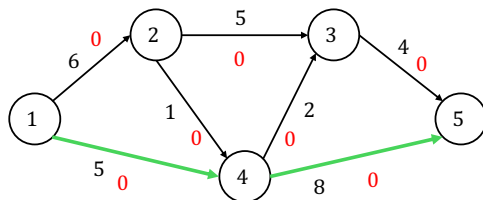
Iterazione : 4
 espando da vertice 1
 raggiungo avanti : 2 theta 1
 espando da vertice 2
 raggiungo avanti : 3 theta 1
 espando da vertice 3

non trovato cammino aumentante,
 vertici raggiunti: 1 2 3

Flusso Massimo = 10

Algoritmo di Edmonds-Karp: Esempio 2

- Supponiamo che Q sia una coda FIFO



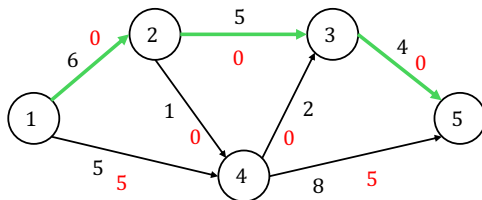
```

Iterazione : 1
espando da vertice 1
raggiungo avanti : 2 theta 6
raggiungo avanti : 4 theta 5
espando da vertice 2
raggiungo avanti : 3 theta 5
espando da vertice 4
raggiungo avanti : 5 theta 5
  
```

```

Pred : 1 1 2 1 4
Theta : Inf 6 5 5 5
trovato cammino aumentante : 5, 4, 1,
valore 5, nuovo flusso 5
  
```


Algoritmo di Edmonds-Karp: Esempio 2 (2)



Iterazione : 2

espando da vertice 1

raggiungo avanti : 2 theta 6

espando da vertice 2

raggiungo avanti : 3 theta 5

raggiungo avanti : 4 theta 1

espando da vertice 3

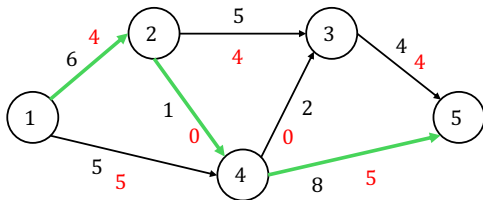
raggiungo avanti : 5 theta 4

Pred : 1 1 2 2 3

Theta : Inf 6 5 1 4

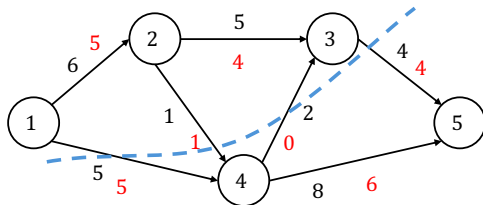
trovato cammino aumentante : 5, 3, 2, 1,
valore 4, nuovo flusso 9

Algoritmo di Edmonds-Karp: Esempio 2 (3)



Iterazione : 3
 espando da vertice 1
 raggiungo avanti : 2 theta 2
 espando da vertice 2
 raggiungo avanti : 3 theta 1
 raggiungo avanti : 4 theta 1
 espando da vertice 3
 espando da vertice 4
 raggiungo avanti : 5 theta 1

Pred : 1 1 2 2 4
 Theta : Inf 2 1 1 1
 trovato cammino aumentante : 5, 4, 2, 1,
 valore 1, nuovo flusso 10



Iterazione : 4
 espando da vertice 1
 raggiungo avanti : 2 theta 1
 espando da vertice 2
 raggiungo avanti : 3 theta 1
 espando da vertice 3

non trovato cammino aumentante,
 vertici raggiunti: 1 2 3

Flusso Massimo = 10

Il Problema del Flusso di Costo Minimo: Algoritmi

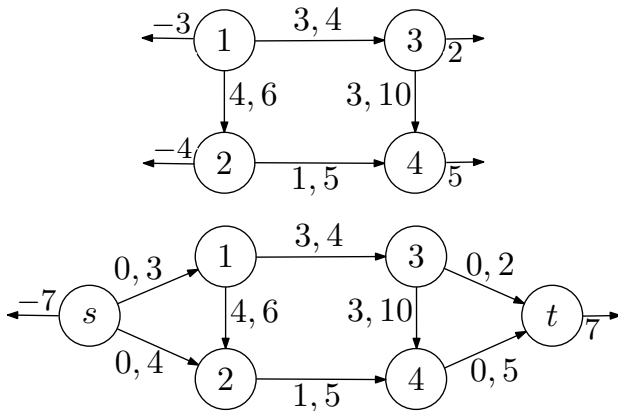
Algoritmi per Flusso di Costo Minimo

- Esistono numerosi algoritmi per determinare il flusso di costo minimo
- Esamineremo dapprima un algoritmo di tipo DUALE (successive shortest paths, cammini minimi successivi) e poi un algoritmo di tipo PRIMALE (cycle cancelling, a cancellazione di cicli)
- Supponiamo che la rete abbia un solo nodo sorgente ed un solo nodo pozzo come discusso precedentemente

Ipotesi semplificativa: una sola sorgente e pozzo

- Spesso, nel progetto di algoritmi per problemi di flusso, conviene assumere che vi sia una sola sorgente e un solo pozzo.
- Si può costruire una rete equivalente:
 - Aggiungendo due nodi fittizi, uno corrispondente all'unica sorgente, e l'altro all'unico pozzo.
 - Aggiungendo archi fittizi dalla sorgente a ciascuna delle sorgenti della rete di partenza. Tali archi avranno costo nullo, e capacità superiore pari all'inverso dello sbilanciamento della sorgente.
 - Aggiungendo archi fittizi da ciascuno dei pozzi della rete di partenza al pozzo. Tali archi avranno anch'essi costo nullo, ma capacità superiore pari allo sbilanciamento del pozzo.
 - Lo sbilanciamento dell'unica sorgente sarà uguale alla somma degli sbilanciamenti delle sorgenti. Similmente per il pozzo.

Ipotesi semplificativa: una sola sorgente e pozzo (2)



Nozioni e Risultati Preliminari — I

- Uno **pseudoflusso** è un vettore x che soddisfa i vincoli di capacità, ossia tale che

$$0 \leq x_{ij} \leq u_{ij} \quad (i,j) \in A$$

- uno pseudoflusso però non soddisfa necessariamente anche i vincoli di bilanciamento
- Se x è uno pseudoflusso, definiamo **sbilanciamento** di un nodo i rispetto a x la quantità

$$e_x(i) = \sum_{(j,i) \in BS(i)} x_{ji} - \sum_{(i,j) \in FS(i)} x_{ij} - b_i$$

- Possiamo anche vedere e_x come un vettore, detto **vettore degli sbilanciamenti**.

Nozioni e Risultati Preliminari — II

- Dato uno pseudoflusso x , i nodi sbilanciati rispetto a x fanno parte di uno dei seguenti due insiemi:

$$O_x = \{i \in N \mid e_x(i) > 0\};$$

$$D_x = \{i \in N \mid e_x(i) < 0\}.$$

I nodi in O_x sono detti **nodì con eccesso di flusso**, mentre quelli in D_x sono detti **nodì con difetto di flusso**.

- Se $O_x = D_x = \emptyset$, allora x è un flusso ammissibile.
- Lo **sbilanciamento complessivo** di x è definito come:

$$g(x) = \sum_{i \in O_x} e_x(i) = - \sum_{j \in D_x} e_x(j)$$

Cammini Aumentanti — I

- Quando si lavora con pseudoflussi, la nozione stessa di cammino aumentante diventa più generale.
- La nozione di **grafo residuo** G_x per uno pseudoflusso x si generalizza banalmente al problema MCF. Ogni arco, però, avrà ora anche un costo:
 - in un arco concorde (i,j) di G_x , il costo è semplicemente c_{ij} ;
 - in un arco discorde (j,i) di G_x , il costo è invece $-c_{ij}$.
- Un cammino P tra i e j in G_x verrà quindi detto **cammino aumentante** tra i e j .
 - I suoi archi possono essere partizionati in P^+ e P^- .
 - La sua capacità $\theta(P,x)$ è definita come per MaxFlow.
 - Un cammino aumentante tra i e i viene anche detto **ciclo aumentante**

Cammini Aumentanti — II

- Dato uno pseudoflusso x e un cammino aumentante P , è possibile inviare $0 \leq \theta \leq \theta(P, x)$ unità di flusso lungo P , attraverso l'operazione $x(P, \theta)$, che conosciamo già
 - In questo contesto, $x(P, \theta)$ verrà spesso indicato anche con $x \oplus P\theta$.
- Se P è un cammino aumentante da i a j in G_x , allora lo pseudoflusso $x(P, \theta)$ avrà gli stessi sbilanciamenti di x , tranne in i e in j .
 - Se $i = j$, allora il vettore degli sbilanciamenti resterà addirittura *inalterato*.
- Il **costo** di un cammino aumentante P è definito come

$$c(P) = \sum_{(i,j) \in P^+} c_{ij} - \sum_{(i,j) \in P^-} c_{ij}$$

- Si verifica facilmente che

$$c \cdot (x(P, \theta)) = c \cdot (x \oplus P\theta) = c \cdot x + \theta c(P).$$

Teorema (Struttura degli Pseudoflussi)

Siano x e y due pseudoflussi qualunque. Allora esistono $k \leq n + m$ cammini aumentanti $P_1, \dots, P_k$, tutti per x , di cui al più m sono cicli, tali che

$$z_1 = x$$

$$z_{i+1} = z_i \oplus \theta_i P_i \quad 1 \leq i \leq k$$

$$z_{k+1} = y$$

$$0 \leq \theta_i \leq \theta(P_i, z_i).$$

Inoltre, tutti i P_i hanno come estremi dei nodi in cui lo sbilanciamento di x è diverso da quello di y .

Pseudoflussi Minimali — I

- A differenza di quello che succede in MF, in MCF non possiamo permetterci di aumentare il flusso indiscriminatamente.
- Un problema centrale è quindi quello di determinare *quali siano* le operazioni di aumento lecite e quali siano le proprietà sui flussi che esse garantiscano.
- Centrale da questo punto di vista è la nozione di **pseudoflusso minimale**, che è uno pseudoflusso x che abbia costo *minimo tra tutti* gli pseudoflussi aventi lo stesso vettore di sbilanciamento e_x .

Pseudoflussi Minimali — II

Lemma

Uno pseudoflusso (rispettivamente, un flusso ammissibile) è minimale (rispettivamente, ottimo) sse non esistono cicli aumentanti di costo negativo.

Pseudoflussi Minimali — II (2)

Dimostrazione.

- ⇒ Per contrapposizione: se esiste un ciclo aumentante di costo negativo in G_x , applicarlo fa diminuire il costo senza alterare lo sbilanciamento, in contraddizione con la minimalità di x .
- ⇐ Ancora per contrapposizione: supponiamo che x non sia minimale, ossia che esista y con $c y < c x$ e $e_y = e_x$. Allora per il teorema sugli pseudoflussi possiamo scrivere $y = x \oplus \theta_1 P_1 \oplus \dots \oplus \theta_n P_n$, dove $\theta_i > 0$ e ciascun P_i è un ciclo. Da $c y < c x$ discende però che:

$$c x > c x + \theta_1 c(P_1) + \dots + \theta_n c(P_n)$$

e quindi che $c(P_i) < 0$ per qualche i .



Pseudoflussi Minimali — III

Teorema

Sia x uno pseudoflusso minimale e sia P un cammino aumentante rispetto ad x avente costo minimo tra tutti i cammini che uniscono un nodo di O_x ad un nodo di D_x . Allora, qualunque sia $\theta \leq \theta(x, P)$, abbiamo che $x(\theta, P) = x \oplus \theta P$ è ancora pseudoflusso minimale.

Pseudoflussi Minimali — III (2)

Dimostrazione.

- Siano s e t i vertici che P collega. Supponiamo che $\theta \leq \theta(x, P)$ e che y sia un qualunque pseudoflusso con vettore di sbilanciamento $e_{x(\theta, P)}$.
- Per il Teorema sulla struttura degli pseudoflussi esistono:
 - k cammini aumentanti $P_1, \dots, P_k$ rispetto a x , tutti da s a t ;
 - h cicli aumentanti $C_1, \dots, C_h$ rispetto a x .

tali che $y = x \oplus \theta_1 P_1 \oplus \dots \oplus \theta_k P_k \oplus \mu_1 C_1 \oplus \dots \oplus \mu_h C_h$, (dove tutti gli θ_i, μ_j sono positivi).

- Deve essere , per ragioni che hanno a che fare con lo sbilanciamento, che $\sum_{1 \leq i \leq k} \theta_i = \theta$.
- Poiché x è minimale, $c(C_i) \geq 0$.
- Siccome P ha costo minimo, $c(P_i) \geq c(P)$.
- Di conseguenza:

$$\begin{aligned} cy &= cx + \theta_1 c(P_1) + \dots + \theta_k c(P_k) + \mu_1 c(C_1) + \dots + \mu_h c(C_h) \\ &\geq cx + \theta c(P) = cx(\theta, P). \end{aligned}$$



Alcuni Algoritmi Ausiliari

- È abbastanza facile costruire uno pseudoflusso minimale x , se non si bada agli sbilanciamenti. Ad esempio, il flusso x definito come

$$x_{ij} = \begin{cases} 0 & \text{se } c_{ij} \geq 0 \\ u_{ij} & \text{altrimenti} \end{cases}$$

In questo modo i costi degli archi nel G_x iniziale sono tutti non-negativi e quindi G_x non avrà cicli negativi (e in ultima analisi x sarà minimale).

- Determinare un cammino di costo minimo tra O_x e D_x è semplice: basta usare uno degli algoritmi basati sui cammini minimi.
- Poichè nel generico G_x ci possono essere archi a costo negativo si può usare Bellman-Ford.

Alcuni Algoritmi Ausiliari (2)

- E' possibile modificare la definizione dell'algoritmo per garantire che i costi in G_x siano sempre non negativi e quindi si possa utilizzare Dijkstra
- Inoltre se la rete ha una sola sorgente ed un solo pozzo ad ogni iterazione si ha che $|O_x| = |D_x| = 1$

CAMMINI MINIMI SUCCESSIVI(G)

1. $x \leftarrow \text{PSEUDOFLUSSO MINIMALE}(G)$;
2. Se $g(x) = 0$, allora termina e restituisci x ;
3. Cerca un cammino di costo minimo P tra un nodo di $i \in O_x$ e $j \in D_x$;
se non esiste, termina: il problema è vuoto;
4. $x \leftarrow x(P, \min\{\theta(P, x), e_x(i), -e_x(j)\})$;
5. Torna al punto 2.

Algoritmo di Bellman-Ford su G_x

- Si genera $G_x = (N, A_x)$ dalla rete e dallo pseudoflusso x corrente.
- In G_x per ogni arco tra i e j (se più di uno) si può includere in A_x solo quello a costo minimo.
- Si associa ad ogni vertice $i \in N$ due etichette:
 - π_i : predecessore di i nel cammino da s ,
 - d_i : lunghezza del cammino corrente da s a i .

Algoritmo di Bellman-Ford su G_x

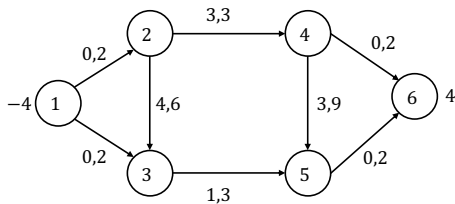
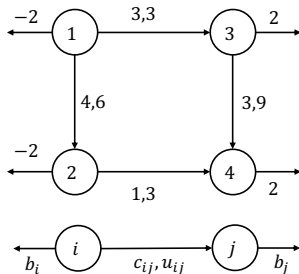
CAMMINO_AUMENTANTE($G_x, s, t, found, \pi, d$)

1. **for each** $i \in N$ **do** $\pi_i = nil, d_i = +\infty$
2. $\pi_s = s, d_s = 0$;
3. **for** $k = 1$ **to** n **do**
4. $Update = False$;
5. **for each** $(i, j) \in A_x$ **do**
6. **if** $d_j > d_i + c_{ij}$ **then**
7. $\pi_j = i, d_j = d_i + c_{ij}$;
8. $Update = True$;
9. **end if**
10. **end for**
11. **if** $Update = True$ **then break**;
12. **end for**
13. **if** $d_t < +\infty$ **then** $found = True$;

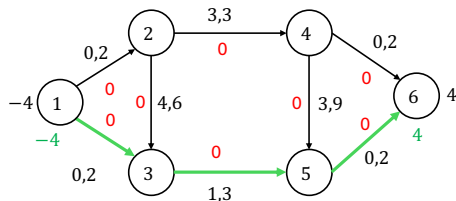
Cammino Aumentante di costo minimo in G_x

- Nella ricerca del cammino aumentante non è necessario costruire esplicitamente G_x
- è sufficiente associare ad ogni vertice $i \in N$ una ulteriore etichetta:
 - θ_i : massimo flusso aggiuntivo inviabile fino a i
 - inoltre, come per il flusso massimo il segno di π_i indica se l'etichettamento avviene con un arco avanti (concorde) o indietro (discorde)
 - se $\pi_i > 0$, usato l'arco avanti (π_i, i) , $\theta_i = \min\{\theta_{\pi_i}, u_{\pi_i, i} - x_{\pi_i, i}\}$
 - se $\pi_i < 0$, usato l'arco indietro $(i, |\pi_i|)$, $\theta_i = \min\{\theta_{|\pi_i|}, x_{i, |\pi_i|}\}$
- ovviamente gli etichettamenti in avanti o indietro devono essere compatibili con l'esistenza degli archi corrispondenti in G_x , ossia dato i
 - si può usare l'arco avanti (i, j) se $u_{ij} - x_{ij} > 0$
 - si può usare l'arco indietro (j, i) se $x_{ji} > 0$

Cammini Minimi Successivi — Esempio

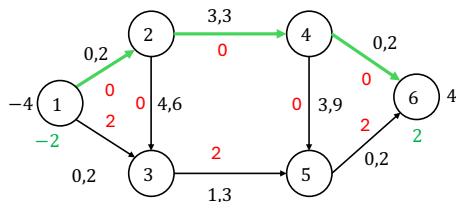


Camminimi Minimi Successivi — Esempio (2)



Iterazione : 1

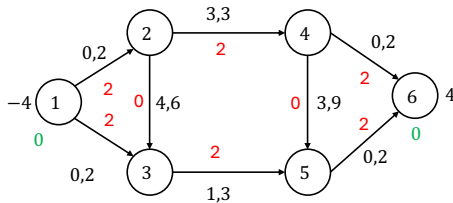
```
nodo 2, (dist,pred,theta) = 0, 1, 2
nodo 3, (dist,pred,theta) = 0, 1, 2
nodo 4, (dist,pred,theta) = 3, 2, 2
nodo 5, (dist,pred,theta) = 1, 3, 2
nodo 6, (dist,pred,theta) = 1, 5, 2
trovato cammino aumentante : 6, 5, 3, 1,
valore 2, nuovo costo 2
```



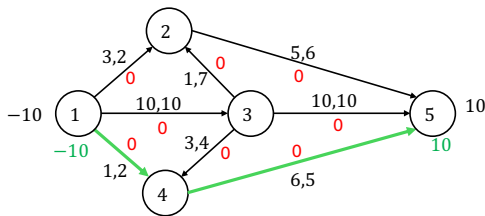
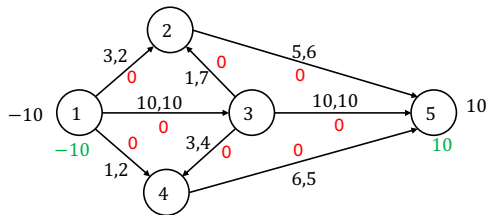
Iterazione : 2

```
nodo 1, (dist,pred,theta) = 0, -1000000, 2
nodo 2, (dist,pred,theta) = 0, 1, 2
nodo 3, (dist,pred,theta) = 2, -5, 2
nodo 4, (dist,pred,theta) = 3, 2, 2
nodo 5, (dist,pred,theta) = 3, -6, 2
nodo 6, (dist,pred,theta) = 3, 4, 2
trovato cammino aumentante : 6, 4, 2, 1,
valore 2, nuovo costo 8
```

Cammini Minimi Successivi — Esempio (3)



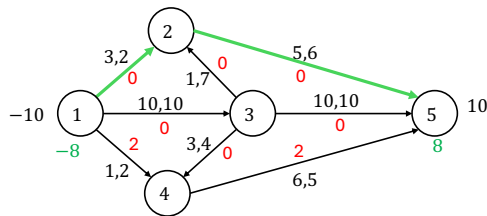
Cammini Minimi Successivi — Esempio 2.19



Iterazione : 1

nodo 1, (dist,pred,theta) = 0, -1000000, 10
 nodo 2, (dist,pred,theta) = 3, 1, 2
 nodo 3, (dist,pred,theta) = 10, 1, 10
 nodo 4, (dist,pred,theta) = 1, 1, 2
 nodo 5, (dist,pred,theta) = 7, 4, 2
 trovato cammino aumentante : 5, 4, 1,
 valore 2, nuovo costo 14

Cammini Minimi Successivi — Esempio 2.19 (2)



Iterazione : 2

nodo 1, (dist,pred,theta) = 0, -1000000, 8

nodo 2, (dist,pred,theta) = 3, 1, 2

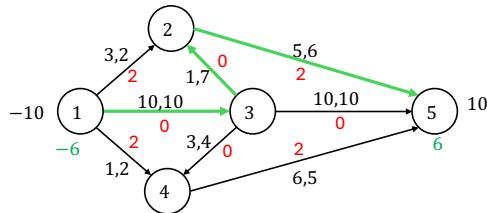
nodo 3, (dist,pred,theta) = 10, 1, 8

nodo 4, (dist,pred,theta) = 2, -5, 2

nodo 5, (dist,pred,theta) = 8, 2, 2

trovato cammino aumentante : 5, 2, 1,

valore 2, nuovo costo 30



Iterazione : 3

nodo 1, (dist,pred,theta) = 0, -1000000, 6

nodo 2, (dist,pred,theta) = 11, 3, 6

nodo 3, (dist,pred,theta) = 10, 1, 6

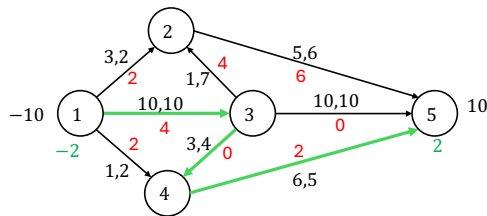
nodo 4, (dist,pred,theta) = 10, -5, 2

nodo 5, (dist,pred,theta) = 16, 2, 4

trovato cammino aumentante : 5, 2, 3, 1,

valore 4, nuovo costo 94

Camminimi Minimi Successivi — Esempio 2.19 (3)

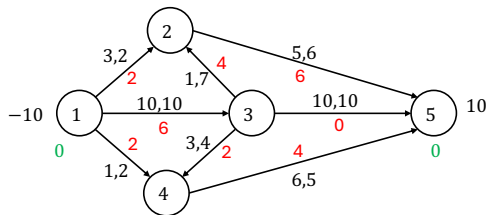


Iterazione : 4

```

nodo 1, (dist,pred,theta) = 0, -1000000, 2
nodo 2, (dist,pred,theta) = 11, 3, 2
nodo 3, (dist,pred,theta) = 10, 1, 2
nodo 4, (dist,pred,theta) = 13, 3, 2
nodo 5, (dist,pred,theta) = 19, 4, 2
trovato cammino aumentante : 5, 4, 3, 1,
valore 2, nuovo costo 132

```



Camminimi Minimi Successivi — Correttezza e Terminazione

- Ad ogni passo, il flusso x rimane minimale.
- **Se l'algoritmo termina**, allora $g(x) = 0$ e quindi x è uno pseudoflusso minimale con sbilanciamento nullo, ossia un flusso di costo minimo.
- Riguardo la **terminazione**, se b e u sono vettori di numeri *interi*, possiamo osservare che:
 - Lo pseudoflusso iniziale è esso stesso intero;
 - Se x è pseudoflusso intero, allora la capacità $\theta(x, P)$ rimane intera;
 - Lo pseudoflusso x rimane quindi *sempre* intero.
 - Ad ogni passo, $g(x)$ diminuisce di almeno 1.

Cammini Minimi Successivi — Complessità

- L'analisi della terminazione che abbiamo appena fatto si estende facilmente anche alla complessità.
- Lo sbilanciamento iniziale $\bar{g}$ è al più

$$\bar{g} \leq \sum_{b_i > 0} b_i + \sum_{c_{ij} < 0} u_{ij}$$

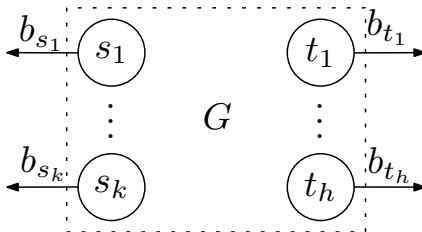
- Come già detto, lo sbilanciamento $g(x)$ cala di almeno 1 ad ogni interazione. Le **iterazioni** saranno quindi al più $\bar{g}$.
- Il costo computazionale di **ogni iterazione** è dominato dalla ricerca di un cammino minimo, che possiamo eseguire in tempo $O(nm)$.
- La complessità sarà quindi nel caso peggiore $O(\bar{g}nm)$, **pseudopolinomiale** nella dimensione del grafo.

Algoritmo a Cancellazione di Cicli

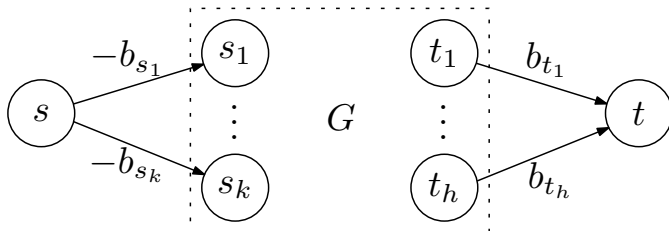
- È un esempio di algoritmo PRIMALE per il flusso di costo minimo
- parte da una soluzione ammissibile e non necessariamente ottima e la migliora iterativamente finchè non prova l'ottimalità della soluzione corrente.

Costruire un Flusso Ammissibile

- Data una rete G , è possibile determinare se esiste un **flusso** ammissibile (ma non necessariamente ottimo) per essa?
- Basta risolvere il problema MF sulla rete ottenuta nel modo seguente



Costruire un Flusso Ammissibile (2)



- Otteniamo così un algoritmo, che chiamiamo $\text{FLUSSOAMMISSIBILE}(G)$, che data una rete, calcola un flusso ammissibile per essa (se esiste).

CANCELLAZIONE CICLI(G)

1. Se FLUSSOAMMISSIBILE(G) restituisce un flusso ammissibile, allora mettilo in x , altrimenti termina: il problema è vuoto.
2. Cerca un ciclo di costo negativo in G_x . Se non lo trovi, allora termina e restituisci x , altrimenti metti il ciclo in C .
3. $x \leftarrow x(C, \theta(C, x))$;
4. Torna al punto 2.

Ricerca Ciclo a Costo Negativo in G_x

- Si genera $G_x = (N, A_x)$ dalla rete e dallo pseudoflusso x corrente.
- In G_x per ogni arco tra i e j (se più di uno) si può includere in A_x solo quello a costo minimo.
- Si associa ad ogni vertice $i \in N$ due etichette:
 - π_i : predecessore di i nel cammino da s ,
 - d_i : lunghezza del cammino corrente da s a i .
- La ricerca del ciclo negativo può essere effettuata con un algoritmo per cammini minimi su grafi con costi anche negativi
- Bellman-Ford se dopo $n - 1$ iterazioni ci sono etichette non ancora ottime ... esiste un ciclo a costo negativo (il cammino ottimo non è semplice)
- Algoritmo di Floyd-Warshall calcola i cammini tra tutte le coppie di vertici in G_x , utilizza due matrici di etichette:
 - d_{ij} lunghezza del cammino minimo da i a j
 - π_{ij} predecessore di j nel cammino minimo da i a j
- al termine se $d_{ii} < 0$ allora esiste un ciclo a costo negativo che coinvolge i

Algoritmo di Floyd-Warshall su G_x

CICLO_DI_COSTO_NEGATIVO($G_x, found, pred, cost$)

1. **for** $i = 1$ **to** n **do**
2. **for** $j = 1$ **to** n **do**
3. **if** $(i, j) \in A_x$ **then**
4. $d_{ij} = c_{ij}, \pi_{ij} = i;$
5. **else**
6. $d_{ij} = +\infty, \pi_{ij} = i;$
7. **end if**
8. **end for**
9. **end for**

Algoritmo di Floyd-Warshall su G_x (2)

% ——— Ricerca cammino

```
10. for  $i = 1$  to  $n$  do
11.   for  $j = 1$  to  $n$  do
12.     for  $k = 1$  to  $n$  do
13.       if  $d_{ij} > d_{ik} + d_{kj}$  then
14.          $d_{ij} = d_{ik} + d_{kj}$ ;  $\pi_{ij} = \pi_{ik}$ ;
15.       end if
16.     end for
17.   end for
18. end for
```

Algoritmo di Floyd-Warshall su G_x (3)

% ——— Individuazione ciclo a costo negativo

```
19. found = False;  
20. for  $i = 1$  to  $n$  do  
21.   if  $d_{ii} < 0$  then  
22.      $found = True, cost = d_{ii}, pred = \pi_{i*};$   
23.     break;  
24.   end if  
25. end for
```

Ciclo a Costo Negativo in G_x

- Nella ricerca del ciclo a costo negativo non è necessario costruire esplicitamente G_x
- è sufficiente definire una ulteriore matrice di etichette:
 - θ_{ij} : massimo flusso aggiuntivo inviabile nel cammino da i a j
 - inoltre, come per il flusso massimo il segno di π_{ij} indica se l'etichettamento avviene con un arco avanti (concorde) o indietro (discorde)
 - se $\pi_{ij} > 0$, usato l'arco avanti (π_{ij}, i) , $\theta_{ij} = \min\{\theta_{\pi_{ij}}, u_{\pi_{ij}.i} - x_{\pi_{ij},i}\}$
 - se $\pi_{ij} < 0$, usato l'arco indietro $(i, |\pi_{ij}|)$, $\theta_{ij} = \min\{\theta_{|\pi_{ij}|}, x_{i,|\pi_{ij}|}\}$
- ovviamente gli etichettamenti in avanti o indietro devono essere compatibili con l'esistenza degli archi corrispondenti in G_x

Cancellazione di Cicli — Proprietà

- La **correttezza** dell'algoritmo è una banale conseguenza del Lemma (che abbiamo dimostrato) sull'equivalenza tra assenza di cicli aumentanti e ottimalità.
- Come al solito, se le capacità sono numeri interi, allora qualcosa diminuisce di almeno 1 ad ogni iterazione, ossia il *costo* (e quindi l'algoritmo **termina**).
- Il costo di qualunque flusso ammissibile è compreso tra $-m\bar{u}\bar{c}$ e $m\bar{u}\bar{c}$, dove

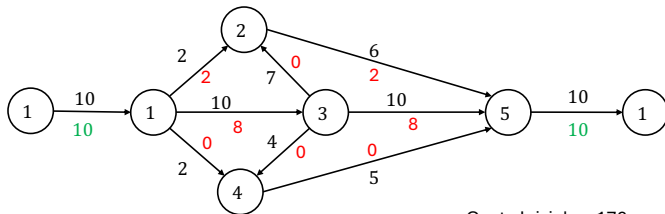
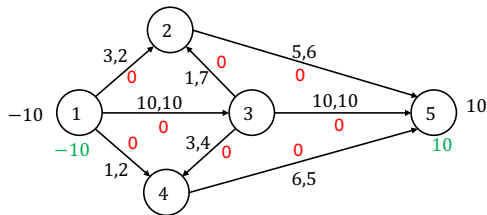
$$\bar{u} = \max\{u_{ij} \mid (i,j) \in N\};$$

$$\bar{c} = \max\{c_{ij} \mid (i,j) \in N\}.$$

La **complessità** dell'algoritmo sarà quindi *pseudopolinomiale*, ossia

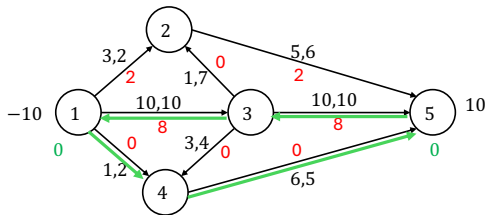
$$O(nm) \cdot O(m\bar{u}\bar{c}) = O(nm^2\bar{u}\bar{c})$$

Cancellazione di Cicli — Esempio 2.19



Costo Iniziale = 176

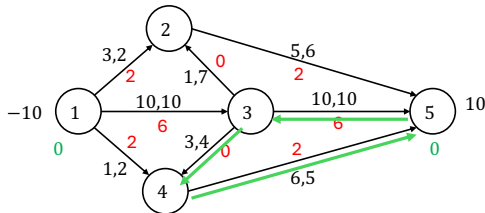
Cancellazione di Cicli — Esempio 2.19 (2)



Iterazione : 1

Trovato ciclo a costo negativo 3, 5, 4, 1, 3,
costo = -13, theta = 2

Nuovo costo = 150

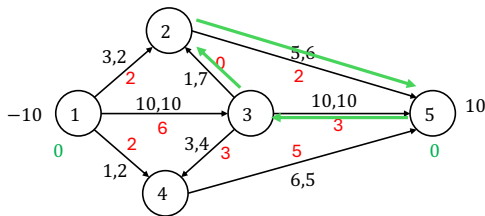


Iterazione : 2

Trovato ciclo a costo negativo 3, 5, 4, 3,
costo = -1, theta = 3

Nuovo costo = 147

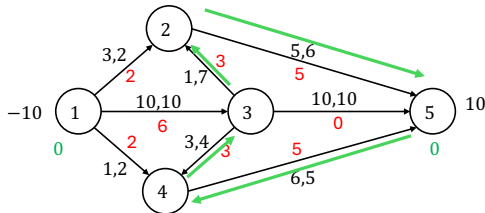
Cancellazione di Cicli — Esempio 2.19 (3)



Iterazione : 3

Trovato ciclo a costo negativo 3, 5, 2, 3,
 costo = -4, theta = 3

Nuovo costo = 135

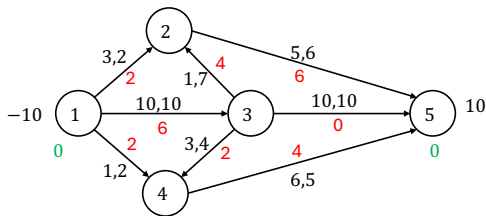


Iterazione : 4

Trovato ciclo a costo negativo 4, 5, 2, 3, 4,
 costo = -3, theta = 1

Nuovo costo = 132

Cancellazione di Cicli — Esempio 2.19 (4)



Iterazione : 5

Non trovato ciclo a costo negativo:
soluzione ottima !

Problemi di Accoppiamento

Nozioni Preliminari — I

- Nei problemi di accoppiamento, si lavora con **grafi bipartiti non orientati** ossia con grafi nella forma $G = (O \cup D, A)$, dove:
 - $O = \{1, \dots, n\}$ è l'insieme dei **nodi origine**;
 - $D = \{n + 1, \dots, n + d\}$ è l'insieme dei **nodi destinazione**;
 - $A \subseteq O \times D$ è l'insieme degli **archi**, a ciascuno dei quali è associato un **costo**.
- Un **accoppiamento** per un grafo bipartito $G = (O \cup D, A)$ è un sottoinsieme M di A i cui archi non abbiano nodi in comune.
 - Gli archi in M si dicono **interni**, mentre quelli in $A - M$ sono detti **esterni**.
 - I nodi che compaiono in qualche arco di M si dicono **accoppiati**, gli altri nodi si dicono invece **esposti**.

Nozioni Preliminari — II

- M è detto **accoppiamento perfetto** sse non vi sono nodi esposti.
- Il *costo* di un accoppiamento M è nient'altro che

$$c(M) = \sum_{(i,j) \in M} c_{ij}.$$

Problemi di Accoppiamento

- **Accoppiamento di Massima Cardinalità**
 - Si vuole determinare, semplicemente, l'accoppiamento di massima cardinalità
- **Accoppiamento di Costo Minimo**
 - Si vuole determinare l'accoppiamento di costo minimo tra tutti gli accoppiamenti *perfetti*.

Accoppiamento di Massima Cardinalità — I

- Il problema può essere visto come un problema di **flusso massimo** con più sorgenti (i nodi in O) e più pozzi (i nodi in D).
 - Le **capacità** saranno tutte pari a 1.
 - Ci interessano solamente **flussi interi**.
 - Come sappiamo, una rete con molte sorgenti e molte destinazioni può essere poi tradotta in una rete con **una** sorgente e **una** destinazione.
- *Flussi ammissibili interi e accoppiamenti* sono in corrispondenza biunivoca.
 - Indicheremo quindi, ad esempio, un flusso con M e il relativo grafo residuo con G_M .

Accoppiamento di Massima Cardinalità — II

- È possibile utilizzare gli algoritmi classici per il problema MF, ma c'è spazio per sfruttare le caratteristiche peculiari dei problemi di accoppiamento.
- A tal proposito osserviamo che ogni cammino aumentante in G_M deve:
 - Essere **alternante**, ossia consistere di archi interni, seguiti da archi esterni, seguiti da archi interni, e così via.
 - Partire da un'origine **esposta** e arrivare ad una destinazione **esposta**.
 - In altre parole, se $P_E = P - M$ e $P_I = M \cap P$ sono rispettivamente gli archi esterni e interni di un cammino aumentante P , allora $|P_E| - |P_I| = 1$.

Accoppiamento di Massima Cardinalità — II (2)

- La capacità $\theta(M, P)$ di un cammino aumentante P sarà sempre 1, e di conseguenza

$$M \oplus (\theta(M, P))P = (M - P_I) \cup P_E.$$

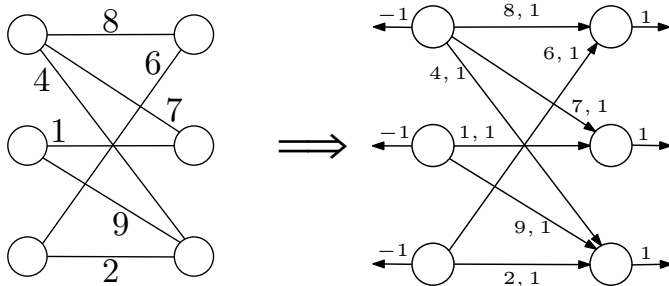
Accoppiamento di Massima Cardinalità — III

- Otteniamo in questo modo un algoritmo del tutto simile a FF, ma in cui la ricerca del cammino aumentante può essere eseguita tramite una semplice procedura di visita.
- La complessità di quest'algoritmo, a differenza di quella di FF, sarà $O(mn)$, perché

$$U = \max\{c_{ij} \mid (i,j) \in A\} = 1.$$

Accoppiamento di Costo Minimo

- Seguendo lo stesso identico tipo di ragionamento, è possibile vedere una qualunque istanza del problema di accoppiamento di costo minimo come un'istanza di MCF.
 - In particolare, i cammini minimi aumentanti potrebbero essere visti come cammini esposti tra due vertici esposti, rispettivamente in O e in D .
- Ad esempio:



Accoppiamento di Costo Minimo (2)